

Elevator Controller Design Documentation

Matthew Wong, Ayden Lee, Darren Zhao

November 23, 2025

Contents

System Description (DR-1)	2
Architecture (DR-2)	3
I/O Map (DR-3)	4
Parameters (DR-4)	5
Trade-offs (DR-5)	5
Limitations and Improvements (DR-6)	6
Appendix	7

System Description (DR-1)

Overview

This project implements a fully digital elevator controller using VHDL on an FPGA. The design models core elevator behavior including request latching, movement scheduling, directional control, floor tracking, door operation, debouncing of inputs, and emergency handling. All timing, such as travel time between floors and door open duration is simulated using counters driven by a 1 Hz clock tick from the 50 MHz FPGA clock.

The controller accepts floor requests from `SW[7..0]`, uses an up/down sweep scheduling policy, and outputs elevator status to LEDs and seven-segment displays. The design emphasizes clarity, deterministic timing, and modular structure while remaining parameterizable for future expansion.

End-to-End Operation

The system begins in the `INITIAL` state, resetting all internal registers and setting the elevator at floor 1. Floor requests are entered through `SW[7..0]`. A built-in debouncing mechanism samples switches at fixed intervals and latches new button presses into the internal request buffer (`req_latched`), where they remain active until served.

The scheduler logic examines whether requests exist above, below, or at the current floor. When idle, the elevator selects a direction based on pending requests. During movement (`MOVE_UP` or `MOVE_DOWN`), a travel timer simulates the up and down motion of an elevator, and when it stops, the elevator updates its floor counter and proceeds to arrival processing. If a request matches the current floor, the controller enters `ARRIVE`, waits briefly, then transitions to `DOOR_OPEN`. While the door is open, the request for that floor is cleared. The controller then enters `DOOR_CLOSE`, evaluates remaining requests, and either continues in the same direction, reverses, or returns to `IDLE`.

Emergency stop (`estop_n = 0`) immediately forces the FSM into `ESTOP`. A soft reset clears only requests, while a hard reset reinitializes the entire system.

Design Assumptions

- Floors are equally spaced with constant travel time.
- No physical sensors exist (floor changes are logic-driven).
- Door timing is simulated by counters.
- Switches behave as momentary buttons, latched until the floor is served.
- Only one elevator is modeled.
- Elevator can hold an infinite number of people.

Architecture (DR-2)

Block Diagram

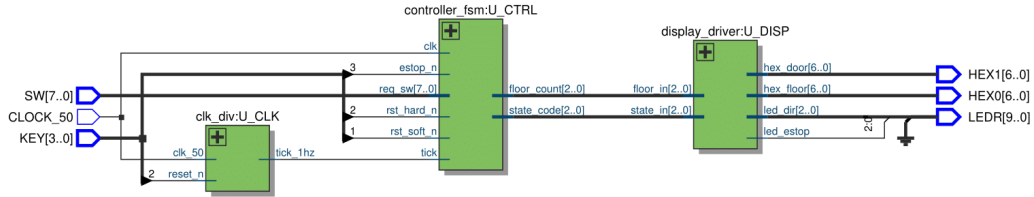


Figure 1: RTL Viewer: Top-level entity `elevator_top`.

Module Descriptions

`controller_fsm`

This module handles elevator logic:

- Debouncing: Turns SW to toggles that unlatches when the request has been served.
- Request latching: Detects rising edges and updates `req_latched`.
- Scheduling logic: Determines `req_above`, `req_below`, `req_here`.
- Movement FSM: INITIAL, IDLE, MOVE_UP, MOVE_DOWN, ARRIVE, DOOR_OPEN, DOOR_CLOSE, ESTOP.
- Timers: Uses `TRAVEL_TIME`, `ARRIVAL_WAIT_TIME`, and `DOOR_OPEN_TIME`.

`display_driver`

Maps states to LEDs and 7-segment displays:

- Converts `floor_in` to HEX0.
- Converts `state_in` to door indicator HEX1.
- Drives direction LEDs `LEDR[2:0]`.
- Drives emergency stop LED `LEDR[9]`.

`clk_div`

Generates 1 Hz tick from 50 MHz input clock using a synchronous counter.

`elevator_top`

Instantiates all modules and connects I/O for the FPGA board.

FSM State Diagram

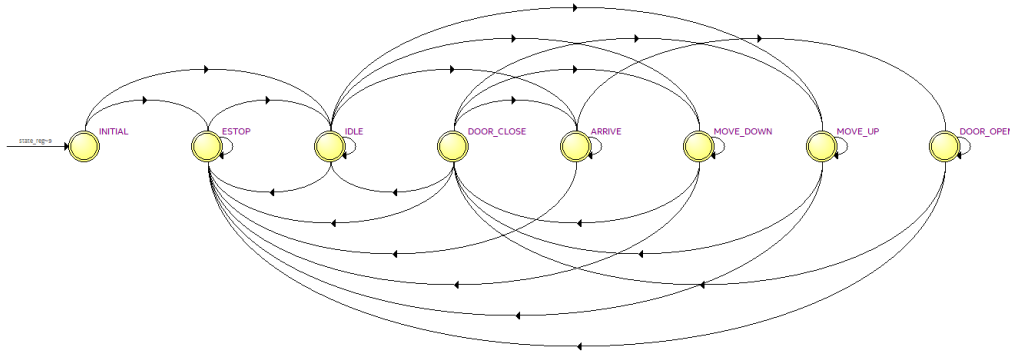
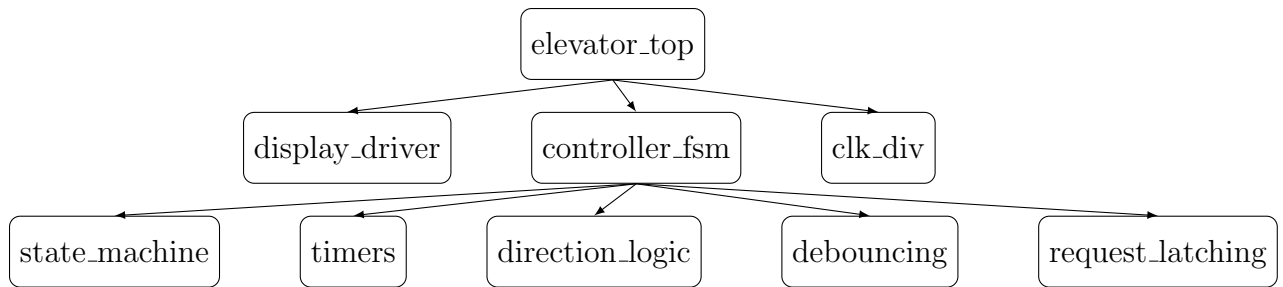


Figure 2: Controller FSM state diagram.

Hierarchy Tree



I/O Map (DR-3)

Input Mapping

Input	Width	Function
SW[7:0]	8	Floor request inputs
KEY[3]	1	Emergency stop (active-low)
KEY[2]	1	Hard reset (active-low)
KEY[1]	1	Soft reset (active-low)
CLOCK_50	1	50 MHz system clock input

Table 1: Input Map

Output Mapping

Output	Width	Function
HEX0	7	Displays current floor
HEX1	7	Displays door/open state
LEDR[2:0]	3	Direction LEDs (up/neutral/down)
LEDR[9]	1	Emergency stop indicator

Table 2: Output Map

Parameters (DR-4)

Parameter Table

Parameter	Default	Description
FLOORS	8	Number of supported floors
TRAVEL_TIME	2 s	Time between floors
ARRIVAL_WAIT_TIME	2 s	Delay before door opens
DOOR_OPEN_TIME	2 s	Duration door remains open

Table 3: Parameter Configuration

Justification

FLOORS = 8 matches the eight physical switches SW[7:0] on the FPGA. Timing values use whole seconds for clear, observable behavior during demonstration. Each parameter allows for easy scaling for different elevator configurations.

Trade-offs (DR-5)

For this elevator controller, a Moore finite-state machine was selected due to its inherently safer and more predictable behavior in safety-critical systems. In a Mealy machine, outputs depend directly on both the current state and real-time inputs. While this can lead to faster reaction times, it also introduces the risk of instantaneous, input-driven output changes, which may cause unsafe or unstable behavior in an elevator. For example, a mechanical switch bounce, a transient signal, or a momentary glitch could cause the Mealy FSM to assert a motor command, reverse direction, or unexpectedly open the doors for a single cycle. In an environment that moves people, such instantaneous reactions are extremely dangerous.

A Moore machine avoids these hazards by basing all outputs solely on the current state, not on synchronous input edges. Every transition into a new operational mode, moving, opening doors, stopping, idling occurs only after the FSM intentionally enters a defined state on the next clock. This creates a controlled response time that guarantees stability even when inputs are complex or invalid. Since state outputs change only on clock boundaries, the asynchronous design inherently filters unsafe instantaneous actions.

Furthermore, Moore FSMs simplify verification: each state corresponds to a well-defined set of actuator outputs, enabling straightforward testing, simulation, and safety auditing. For a virtual elevator—where passenger safety, predictable motion, and deterministic timing are essential—the Moore approach ensures that motion, scheduling decisions, and door operations occur only through deliberate, validated state transitions rather than instantaneous input-driven reactions.

The scheduling uses an up/down sweep algorithm. When idle, the elevator selects a direction based on queued requests, continues in that direction, and reverses only after all requests in that direction are served.

Parameterization using generics such as `FLOORS` and `TRAVEL_TIME` makes the design scalable. Separating functionality across `clk_div`, `controller_fsm`, and `display_driver` provides modularity and makes the system easier to test and debug.

Limitations and Improvements (DR-6)

Current Limitations

- No physical motion sensors; floor changes are logic based.
- Only supports one elevator car.
- No overload detection.
- Constant travel speed.

Future Improvements

- Add physical sensors and full-capacity detection.
- Implement multi-car elevator.
- Add call button lights.

Appendix

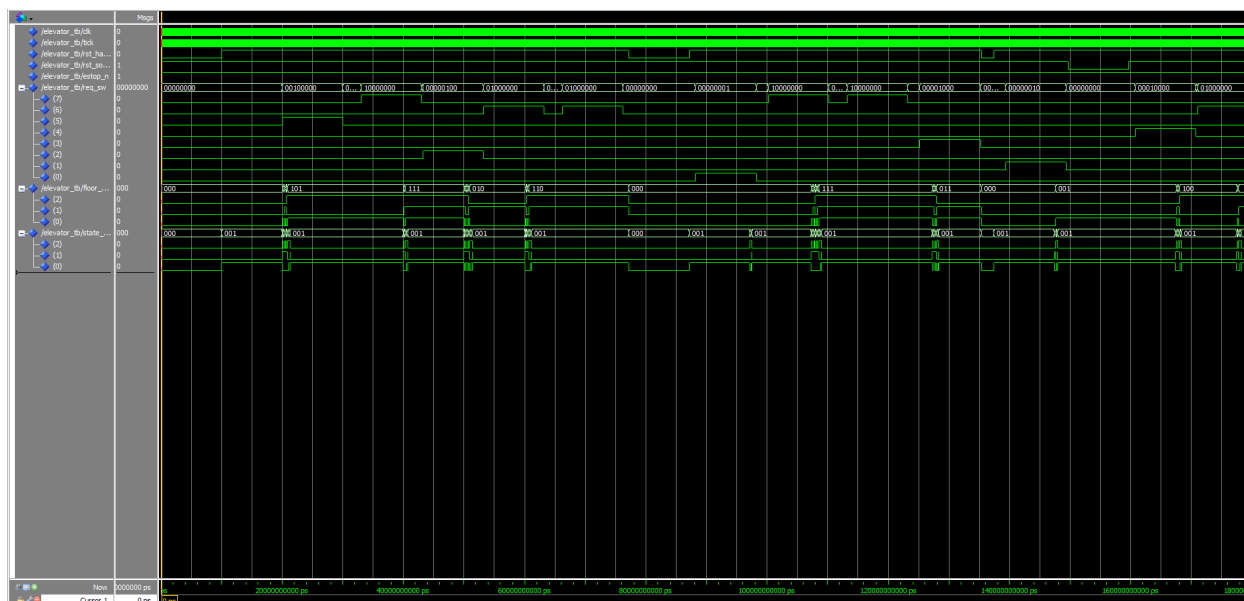


Figure 3: Timing diagram for FSM test-bench

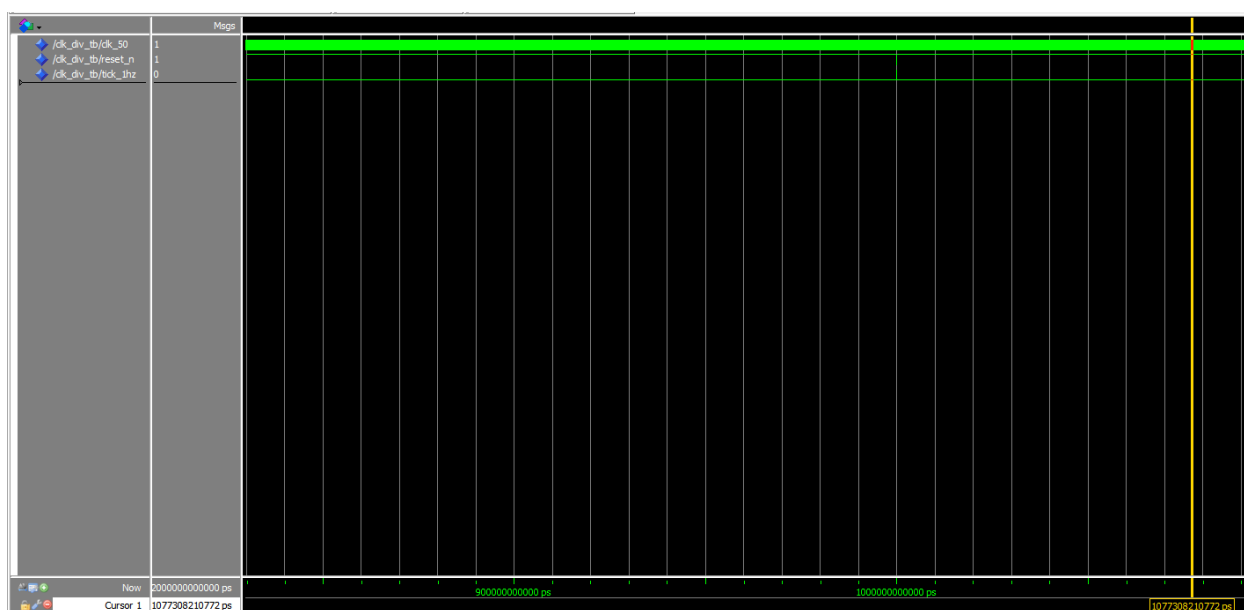


Figure 4: Timing diagram for clock division test-bench

Table 4: Truth table for FSM test-bench

Step	Inputs	Case	Expected Behavior
1	<code>rst_hard_n = 0</code>	Hard reset asserted	State $\rightarrow$ INITIAL, floor = 1
2	<code>rst_hard_n = 1</code>	Hard reset released	Enter IDLE
3	<code>req_sw(5)=1 then 0</code>	Request floor 5	Move to floor 5, arrive, open doors
4	<code>req_sw(7)=1 then 0</code>	Request above current floor	Begin upward movement
5	<code>req_sw(2)=1 then 0</code>	Request below while moving up	Queue request, maintain upward priority
6	<code>req_sw(6)=1 then 0</code>	New request above while moving	Continue moving up
7	<code>req_sw(current)=1 then 0</code>	Request equals current floor	Immediate ARRIVE $\rightarrow$ DOOR_OPEN
8	<code>rst_hard_n = 0 $\rightarrow$ 1</code>	Hard reset at boundary	Reset floor to 0, state $\rightarrow$ IDLE
9	<code>req_sw(0)=1 then 0</code>	Request bottom floor	No movement, open door
10	<code>req_sw(7)=1 then 0</code>	Request top floor	Move to floor 7
11	<code>req_sw(7)=1 then 0</code>	Request same floor again	No movement
12	<code>req_sw(3)=1 then 0</code>	Start movement	Begin MOVE_UP or MOVE_DOWN
13	<code>rst_hard_n = 0 $\rightarrow$ 1</code>	Hard reset mid-motion	Reset to floor 0, IDLE
14	<code>req_sw(1)=1 then 0</code>	Trigger door sequence	ARRIVE $\rightarrow$ DOOR_OPEN
15	<code>rst_soft_n = 0 $\rightarrow$ 1</code>	Soft reset during DOOR_OPEN	Clear requests, remain at floor
16	<code>req_sw(4)=1 then 0</code>	New request	Move to floor 4
17	<code>req_sw(6)=1 during door open</code>	New request during DOOR_OPEN	Queue request
18	<code>estop_n = 0 $\rightarrow$ 1</code>	Emergency stop	Enter ESTOP, then recover
19	<code>rst_soft_n = 0 $\rightarrow$ 1</code>	Exit ESTOP	Return to IDLE
20	<code>req_sw(2)=1, 5=1, 7=1 then clear</code>	Multiple queued requests	Full direction sweep and request handling

Table 5: Truth table for Clock Divider (clk_div) Testbench

Step	Inputs / Condition	Case	Expected Behavior
1	<code>reset_n</code> = 0, $t < 100$ ns	Asynchronous Reset asserted	Internal counter holds 0, <code>tick_1hz</code> stays logic '0'.
2	<code>reset_n</code> = 1	Reset released, Clock active (50 MHz)	Internal counter begins incrementing from 0 on every rising edge.
3	$t \approx 100$ ns to 1.0 s	Counting Phase (Counter < 49,999,999)	<code>tick_1hz</code> remains '0' while counter increments.
4	Counter = 49,999,999	Terminal Count Reached (1 Second elapsed)	<code>tick_1hz</code> asserts '1' for exactly one clock cycle (20 ns).
5	Next Clock Edge	Counter Rollover	Internal counter resets to 0, <code>tick_1hz</code> returns to '0'.
6	End of Simulation (+10000 ns)	Post-tick observation	<code>tick_1hz</code> remains '0' as the new second begins counting.

Contribution Distribution:

Matthew Wong: 33%

Ayden Lee: 33%

Darren Zhao: 33%